IITB Integrated Security System

This manual explains how to clone, prepare, start, and verify the IITB Integrated Security System using seven separate PowerShell terminals on Windows.

Complete Startup Guide

This guide starts the project from a fresh Git clone on Windows PowerShell.

Terminal	Service	Port
1	PostgreSQL database	5432
2	Campus Security backend	8000
3	Campus Security frontend	5000
4	CORTEX application	3000
5	GCS backend API	5002
6	Drone backend API	8001
7	GCS dashboard	8765

`8000` is reserved for the Campus Security backend. The drone backend uses `8001`.

1. Prerequisites

Install and start Git, Docker Desktop, Node.js 18 or newer, and Python 3.10 or newer.

```
git --version
docker --version
docker compose version
node --version
npm --version
py --version
```

If Docker reports a named-pipe or permission error, start Docker Desktop and wait until its Linux engine is running.

2. Clone the repository

```
cd C:\Users\HP
git clone https://github.com/IIT-Bombay-MDM/IITB-Integrated-Security-System.git "IITB Internship"
cd "C:\Users\HP\IITB Internship"
```

If the folder already exists, update it instead:

```
cd "C:\Users\HP\IITB Internship"
git pull
```

3. Install dependencies (first setup only)

Campus backend and GCS backend

```
cd "C:\Users\HP\IITB Internship\campus-security-system-complete\backend_server"
py -3 -m venv venv
.\venv\Scripts\python.exe -m pip install --upgrade pip
.\venv\Scripts\python.exe -m pip install -r requirements.txt
```

The campus virtual environment is also used for the GCS Flask backend.

Drone backend

```
cd "C:\Users\HP\IITB Internship\send_gcs\T1T2IIT(Bom) copy\drone_backend"
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Campus frontend

```
cd "C:\Users\HP\IITB Internship\campus-security-system-complete\frontend"
npm.cmd install
```

CORTEX

```
cd "C:\Users\HP\IITB Internship\send_gcs\T1T2IIT(Bom) copy"
npm.cmd install
```

4. Start the seven terminals

Open seven separate PowerShell windows or tabs. Run one section in each terminal and keep every server terminal open.

Terminal 1: PostgreSQL database, port 5432

```
cd "C:\Users\HP\IITB Internship"
docker compose up db
```

Wait until PostgreSQL is ready to accept connections.

Terminal 2: Campus Security backend, port 8000

```
cd "C:\Users\HP\IITB Internship\campus-security-system-complete\backend_server"
$env:DATABASE_URL="postgresql://campus_security:campus_security@localhost:5432/campus_security"
.\venv\Scripts\uvicorn.exe main:app --host 127.0.0.1 --port 8000
```

Health check: `http://127.0.0.1:8000/health`

Terminal 3: Campus Security frontend, port 5000

```
cd "C:\Users\HP\IITB Internship\campus-security-system-complete\frontend"
npm.cmd run dev
```

Open: `http://localhost:5000`

Terminal 4: CORTEX, port 3000

```
cd "C:\Users\HP\IITB Internship\send_gcs\T1T2IIT(Bom) copy"
npm.cmd run dev
```

Open: `http://localhost:3000`

Terminal 5: GCS backend API, port 5002

```
cd "C:\Users\HP\IITB Internship\send_gcs\GCS_folder\backend"
$env:GCS_BACKEND_PORT="5002"
..\..\..\campus-security-system-complete\backend_server\venv\Scripts\python.exe app.py
```

Health check: `http://localhost:5002/health`

Port 5002 is an API, not a website. Opening `http://localhost:5002/` may show `Not Found` even when the API is working.

Terminal 6: Drone backend API, port 8001

```
cd "C:\Users\HP\IITB Internship\send_gcs\T1T2IIT(Bom) copy\drone_backend"
.\.venv\Scripts\python.exe main.py
```

Open API docs: `http://127.0.0.1:8001/docs`

If port 8001 is already in use, do not start a second copy. Verify the existing server:

```
curl.exe http://127.0.0.1:8001/docs
```

Terminal 7: GCS dashboard, port 8765

```
cd "C:\Users\HP\IITB Internship\send_gcs"
..\campus-security-system-complete\backend_server\venv\Scripts\python.exe -m http.server 8765 --
directory frontend
```

Open: `http://localhost:8765`

5. Verify all services

Run these checks from a separate PowerShell window:

```
curl.exe http://127.0.0.1:8000/health
curl.exe http://localhost:5002/health
curl.exe http://127.0.0.1:8001/docs
curl.exe -I http://localhost:5000
curl.exe -I http://localhost:3000
curl.exe -I http://localhost:8765
```

Expected results:

- Campus backend returns JSON with `"status":"ok"`.
- GCS backend returns JSON containing service status fields.
- Drone backend returns Swagger UI HTML.
- The three frontend checks return a success or redirect response.

Check the database from the repository root:

```
docker compose ps
```

The `db` container should show `Up` or `running`.

6. Browser addresses

- Campus Security UI: `http://localhost:5000`
- Campus backend health: `http://127.0.0.1:8000/health`
- CORTEX: `http://localhost:3000`
- GCS dashboard: `http://localhost:8765`
- GCS backend health: `http://localhost:5002/health`
- Drone API docs: `http://127.0.0.1:8001/docs`

7. Common problems

Port already in use

```
Get-NetTCPConnection -LocalPort 8001 -State Listen
Get-Process -Id PROCESS_ID
```

A port conflict often means that service is already running. Verify it before stopping anything.

Docker does not start

Start Docker Desktop, then run `docker compose up db` from the repository root.

Do not mix startup methods

Do not run `send_gcs\start-gcs.ps1` together with Terminals 4, 5, 6, and 7. The launcher starts GCS services automatically and can create duplicate port conflicts.

8. Stop the system

Press `Ctrl+C` in each server terminal, including the Docker terminal. PostgreSQL data remains in its Docker volume unless the volume is explicitly removed.